PROJECT SUBMISSION DOCUMENT

Veritas AI: Fake News Detection System Frontend Architecture & Source Code

Complete technical documentation and source code implementation of the Web Authentication Portal, Responsive UI, and Streamlit Dashboard.

Project Title:	Veritas AI - Fact-Checking & Misinformation Detection
Module:	Frontend UI, Authentication, and User Experience Layer
Tech Stack:	HTML5, CSS3, PHP, MySQL, Streamlit UI
Accepted Formats:	PDF (.pdf), Word 2007 (.docx), Word (.doc)

1. Frontend Architecture Overview

The Veritas AI user interface is architected around a dual-layer interaction model:

- **Authentication & User Gate Portal:** Built with responsive semantic HTML5, CSS3 styling, and PHP database connectivity (`connection.php`, `authen1.php`, `authen3.php`). It validates user credentials against a MySQL customer table, maintains session state in a current activity table, and seamlessly routes authorized users to the analytics engine.
- **Interactive Veritas AI Streamlit Dashboard:** A high-performance Python frontend providing claim submission input, real-time evidence retrieval progress, visual probability gauges, sentence-level NLI entailment breakdowns, and source transparency tables.

File Name	Role / Component	Language / Technology
login.html	User Login Interface	HTML5, Form Handling
register.html	New User Registration	HTML5, Form Handling
style.css	Visual Design & Responsive Layout	CSS3, Flexbox/Grid
connection.php	Database Connection Handler	PHP, MySQLi Driver
authen1.php	Authentication & Session Dispatch	PHP, SQL Sanitization
authen3.php	User Onboarding & Account Creation	PHP, Prepared Statements
streamlit_factcheck.py	Interactive Fact-Checking Dashboard	Python, Streamlit Component Model

2. Web Portal Interface: login.html

User Login Page login.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Login Form</title>
    <link rel="stylesheet" type="text/css" href="style.css">
</head>
<style>
body {
  background-image: url('bg3.png');
 background-repeat: no-repeat;
background-attachment: fixed;
  background-size: cover;
}
</style>

<body>
  <div class="header">
    <h1 style="margin-top: 0px;color: aliceblue">FAKE NEWS DETECTION SYSTEM</h1>
</div>
  <h2 style="font-size:35px;position:absolute; left:600px; top:100px;font-family:coldiac;">LOGIN</h2>
<br>
  <div class="login" style = "position:absolute; left:10px; top:200px; background-color:white; border-
color:black;">
    <form id="Login" method="POST" action="authen1.php">
        <label style="text-align:center; margin-bottom:5 px"><b>Username
        </b>
        </label>
        <input type="text" style="margin-bottom:2px" name="Uname" id="Uname" placeholder="username">
        <br><br>
        <label><b>Password
        </b>
        </label>
        <input type="Password" name="Pass" id="Pass" placeholder="password">
        <br><br>
        <input type="submit" style="border-color:rgb(13, 50, 83) ; margin-left:10px" name="log"
id="log" value="Log In">
        <br><br>
        <input type="submit" style="border-color:rgb(13, 50, 83) ; margin-left:10px" name="reg"
id="reg" value="Create New Account">
        <br><br>



    </form>
</div>
</body>
</html>
```

3. User Registration Interface: register.html

User Registration Page register.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Login Form</title>
    <link rel="stylesheet" type="text/css" href="style.css">
</head>
<style>
body{
  background-image:url('bg3.png');
 background-repeat: no-repeat;
background-attachment: fixed;
  background-size: cover;
}
</style>

<body>
  <div class="header">
    <h1 style="margin-top: 0px;color: aliceblue;">FAKE NEWS DETECTION SYSTEM</h1>
</div>
  <h2 style="font-size:35px;position:absolute; left:580px; top:100px;font-
family:coldiac;">REGISTER</h2><br>
  <div class="login" style = "position:absolute; left:10px; top:200px; background-color:white; border-
color:black;">
    <form id="Lreg" method="POST" action="authen3.php">
        <label style="text-align:center; margin-bottom:5 px"><b>Username
        </b>
        </label>
        <input type="text" style="margin-bottom:2px" name="Uname" id="Uname" placeholder="username">
        <br><br>
        <label><b>Password
        </b>
        </label>
        <input type="Password" name="Pass" id="Pass" placeholder="password">
        <br><br>
        <input type="submit" style="border-color:rgb(13, 50, 83);padding-bottom:5px; margin-
top:25px;margin-left:10px" name="reg" id="reg" value="Create Account">
        <br><br>


    </form>
</div>
</body>
</html>
```

4. Stylesheet & Design Tokens: style.css

Frontend Stylesheet style.css

```
body
{
    margin: 0;
    padding:0;
    background-color:#F8F8FF;
    font-family: 'Arial';
}
.header
  {
  text-align: center;
  background: #000000;
  font-size: 35px;
  font-family: 'Times New Roman', Times, serif;
  margin-top:0px;
  height:80px;
  padding: bottom 8px;
  padding-top:7px;
  }
.login{
        margin-left:490px;
        width: 210px;
        overflow: hidden;
        margin: 30 0 0 300px;
        padding: 80px;
        padding-bottom:30px;
        padding-top:30px;
        padding-left:30px;
        background: #F8F8FF;
        border-radius: 15px ;
        border-style: solid;
        border-color:rgb(95, 155, 208);

}
h2{
    text-align: center;
    color: #000000;
    padding-top:10px;
    color: beige;

}
label{
    color: #000000;
    text-align: center;
    font-size: 17px;
}
#Uname{
    width: 210px;
    height: 30px;
    border: none;
    border-radius: 3px;
    padding-left: 8px;
    padding-right: 8px;
}
#Pass{
    width: 210px;
    height: 30px;
    border: none;
    border-radius: 3px;
    padding-left: 8px;
    padding-right: 8px;

}
#log{
```

```
    width:220px;
    height: 30px;
    border: none;
    border-radius: 17px;
    padding-left: 7px;
    padding-right: 7px;
    color: black;
    background-color:rgb(95, 155, 208);


}
#reg{
    width:220px;
    height: 30px;
    border: none;
    border-radius: 17px;
    padding-right: 7px;
    padding-left: 7px;
    color: black;
    background-color:rgb(95, 155, 208) ;

}
span{
    border-color:rgb(95, 155, 208)  ;
    font-size:15px;
}
a{
    float: right;
    border-color:rgb(95, 155, 208)
}
```

5. Database Connection Handler: connection.php

Database Bridge connection.php

```
<?php
    // $host = "localhost";
    // $user = "root";
    // $password = "";
    // $db_name = "mydatabase";

    $host = "remotemysql.com";
    $user = "ywNaDEUMs0";
    $password = "z3FwsBlKs8";
    $db_name = "ywNaDEUMs0";

    $con = mysqli_connect($host, $user, $password, $db_name);
    if(mysqli_connect_errno()) {
        die("Failed to connect with MySQL: ". mysqli_connect_error());
    }
?>
```

6. Authentication & Session Dispatcher: authen1.php

Auth Validator & Redirect authen1.php

```
<?php
    include('connection.php');

    $username = $_POST['Uname'];
    $password = $_POST['Pass'];
    $r="1";

        //to prevent from mysqli injection
        $username = stripcslashes($username);
        $password = stripcslashes($password);
        $username = mysqli_real_escape_string($con, $username);
        $password = mysqli_real_escape_string($con, $password);
        $number1=1;
        if(isset($_POST['log'])){
        if(!empty($_POST['Uname']) && !empty($_POST['Pass']))
    {
        $sql = "select *from ywNaDEUMs0.customer where username = '$username' and password1 =
'$password'";
        $result = mysqli_query($con, $sql);
        $row = mysqli_fetch_array($result, MYSQLI_ASSOC);
        $count = mysqli_num_rows($result);

        if($count >= 1){
            header("location: http://localhost:8501");
            $query="INSERT INTO ywNaDEUMs0.current VALUES('$username','$r')";
            $result=mysqli_query($con,$query);
            echo "<h1><center> Login successful </center></h1>";

        }
        else{
             echo '<script>alert("Login Unsuccessful! Invalid Username / Password: Please try again")
</script>';
             }
     }else{echo '<script>alert("Fields are left empty! Try again.")</script>';}

      }

      if(isset($_POST['reg'])){
      header("location: register.html");}

?>
```

7. User Registration Handler: authen3.php

Account Creator authen3.php

```
<?php
  include('connection.php');
    $username=$_POST['Uname'];
    $password=$_POST['Pass'];

      if(isset($_POST['reg'])){
    $username = stripcslashes($username);
    $password = stripcslashes($password);
    $username = mysqli_real_escape_string($con, $username);
    $password = mysqli_real_escape_string($con, $password);
    if(!empty($_POST['Uname']) && !empty($_POST['Pass']))
    {
        $sql = "select * from ywNaDEUMs0.customer where username = '$username'";
        $result = mysqli_query($con, $sql);
        $row = mysqli_fetch_array($result, MYSQLI_ASSOC);
        $count = mysqli_num_rows($result);
        if($count >= 1){
          echo '<script>alert("Username is already taken, Please try again with a different
username!")</script>';
        }
         else{
             $query="INSERT INTO ywNaDEUMs0.customer(username,password1)
VALUES('$username','$password')";
              $result=mysqli_query($con,$query);
              if($result){
               echo "Account Successfully Created";
               header("location: login.html");
               }
               else {
                   echo "Failure!";
                     }
              }}
           else
           {
              echo '<script>alert("Fields are left empty! Try again.")</script>';
            }

}




?>
```

8. Veritas AI Interactive Dashboard UI: streamlit_factcheck.py

Streamlit Full-Stack UI Application `streamlit_factcheck.py`

```
import contextlib
import io
import json
from datetime import datetime

import streamlit as st

from factcheck_engine.services.fact_checker import FactChecker
from factcheck_engine.services.evidence_store import EvidenceStore


# ------------------------------------------------
# PAGE CONFIGURATION
# ------------------------------------------------

st.set_page_config(
    page_title="Veritas | AI Fact Checker",
    page_icon="🔎",
    layout="wide",
    initial_sidebar_state="expanded",
)


# ------------------------------------------------
# CUSTOM STYLING — WHITE AND RED
# ------------------------------------------------

st.markdown(
    """
    <style>
    @import url('https://fonts.googleapis.com/css2?
family=DM+Sans:wght@400;500;600;700&family=Manrope:wght@400;500;600;700;800&display=swap');


    html, body, .stApp,
    [class*="css"],
    [data-testid="stMarkdownContainer"],
    [data-testid="stText"],
    .stButton button,
    .stTextInput input,
    .stTextArea textarea,
    label,
    p, span, div {
        font-family: "DM Sans", -apple-system, BlinkMacSystemFont,
                     "Segoe UI", sans-serif !important;
    }

    h1, h2, h3, h4, h5, h6 {
        font-family: "Manrope", "DM Sans", sans-serif !important;
        font-weight: 700 !important;
    }

    .hero h1 {
        font-family: "Manrope", sans-serif !important;
        font-weight: 800 !important;
        font-size: 2.5rem !important;
    }

    :root {
        --red: #e53935;
        --red-dark: #c62828;
        --white: #ffffff;
        --black: #171717;
```

```
        --gray: #737373;
        --border: #eeeeee;
    }

    html, body, [class*="css"] {
        font-family: "DM Sans", sans-serif;
        color: var(--black);
    }

    .stApp {
        background: var(--white);
        color: var(--black);
    }

    .block-container {
        max-width: 1200px;
        padding-top: 2rem;
        padding-bottom: 3rem;
    }

    section[data-testid="stSidebar"] {
        background: var(--white);
        border-right: 1px solid var(--border);
    }

    section[data-testid="stSidebar"] h2 {
        color: var(--red);
        font-weight: 800;
    }

    .hero {
        background: var(--white);
        border: 1px solid var(--border);
        border-top: 5px solid var(--red);
        padding: 2.3rem;
        border-radius: 16px;
        margin-bottom: 1.8rem;
    }

    .hero h1 {
        color: var(--black);
        font-size: 2.5rem;
        font-weight: 800;
        margin-bottom: 0.7rem;
        line-height: 1.2;
    }

    .hero p {
        color: var(--gray);
        font-size: 1.05rem;
        line-height: 1.7;
        margin-bottom: 0;
    }

    .section-label {
        color: var(--red);
        font-size: 0.76rem;
        font-weight: 700;
        letter-spacing: 0.13em;
        text-transform: uppercase;
        margin-bottom: 0.7rem;
    }

    .info-card {
        background: var(--white);
        border: 1px solid var(--border);
```

```
        border-radius: 14px;
        padding: 1.2rem;
        margin-bottom: 0.8rem;
        min-height: 145px;
    }

    .info-card h4 {
        color: var(--red);
        margin-top: 0;
    }

    div[data-testid="stMetric"] {
        background: var(--white);
        padding: 1rem;
        border: 1px solid var(--border);
        border-radius: 12px;
    }

    div[data-testid="stMetricValue"] {
        color: var(--red);
        font-weight: 700;
    }

    .stButton > button,
    .stDownloadButton > button,
    .stFormSubmitButton > button {
        border-radius: 9px;
        font-weight: 600;
        min-height: 2.8rem;
    }

    .stButton > button[kind="primary"],
    .stFormSubmitButton > button[kind="primary"] {
        background: var(--red);
        color: var(--white);
        border: 1px solid var(--red);
    }

    .stButton > button[kind="primary"]:hover,
    .stFormSubmitButton > button[kind="primary"]:hover {
        background: var(--red-dark);
        border-color: var(--red-dark);
        color: var(--white);
    }

    .stDownloadButton > button {
        background: var(--white);
        color: var(--red);
        border: 1px solid var(--red);
    }

    .stDownloadButton > button:hover {
        background: var(--red);
        color: var(--white);
    }

    .stTextArea textarea,
    .stTextInput input {
        background: var(--white);
        color: var(--black);
        border: 1px solid #dddddd;
        border-radius: 10px;
    }

    .stTextArea textarea:focus,
    .stTextInput input:focus {
```

```
        border-color: var(--red);
        box-shadow: 0 0 0 1px var(--red);
    }

    div[data-testid="stExpander"] {
        background: var(--white);
        border: 1px solid var(--border);
        border-radius: 12px;
    }

    h1, h2, h3, h4 {
        color: var(--black);
    }

    a {
        color: var(--red) !important;
    }

    footer {
        visibility: hidden;
    }

    /* FIX TEXT CONTRAST — WHITE AND RED THEME */

    section[data-testid="stSidebar"],
    section[data-testid="stSidebar"] * {
        color: #222222;
    }

    section[data-testid="stSidebar"] h1,
    section[data-testid="stSidebar"] h2,
    section[data-testid="stSidebar"] h3 {
        color: #e53935 !important;
    }

    section[data-testid="stSidebar"] [data-testid="stRadio"] label,
    section[data-testid="stSidebar"] [data-testid="stRadio"] label p,
    section[data-testid="stSidebar"] [data-testid="stRadio"] label span {
        color: #222222 !important;
        opacity: 1 !important;
    }

    section[data-testid="stSidebar"] [data-testid="stMetricLabel"],
    section[data-testid="stSidebar"] [data-testid="stMetricLabel"] *,
    section[data-testid="stSidebar"] [data-testid="stMetricValue"],
    section[data-testid="stSidebar"] [data-testid="stMetricValue"] * {
        color: #e53935 !important;
        opacity: 1 !important;
    }

    section[data-testid="stSidebar"] p,
    section[data-testid="stSidebar"] span {
        opacity: 1 !important;
    }

    .stApp,
    .stApp p,
    .stApp label,
    .stApp [data-testid="stMarkdownContainer"] p {
        color: #222222;
    }

    .hero h1,
    h1, h2, h3, h4 {
        color: #171717 !important;
    }
```

```
    .hero p {
        color: #555555 !important;
    }

    .section-label {
        color: #e53935 !important;
    }

    .stCaption,
    [data-testid="stCaptionContainer"] {
        color: #666666 !important;
        opacity: 1 !important;
    }

</style>
    """,
    unsafe_allow_html=True,
)


# ------------------------------------------------
# BACKEND INITIALIZATION
# ------------------------------------------------

@st.cache_resource(show_spinner=False)
def get_fact_checker():
    return FactChecker()


def get_evidence_records():
    try:
        return EvidenceStore().load_all()
    except Exception as exc:
        print(f"Could not load evidence records: {exc}")
        return []


# SIDEBAR
# ------------------------------------------------

with st.sidebar:
    st.markdown("## 🔎 VERITAS")
    st.caption("Evidence-Based Fact Verification")

    st.divider()

    st.markdown("### Navigation")
    page = st.radio(
        "Choose a section",
        ["Fact Checker", "Evidence Store", "About Project"],
        label_visibility="collapsed",
    )

    st.divider()

    records = get_evidence_records()
    verified_count = sum(
        1 for r in records
        if r.get("evidence_status") == "verified"
    )
    pending_count = sum(
        1 for r in records
        if r.get("evidence_status") == "pending_review"
    )
```

```
    st.markdown("### Knowledge Base")
    st.metric("Stored evidence records", len(records))
    st.metric("Verified records", verified_count)
    st.metric("Pending review", pending_count)

    st.divider()

    st.caption("NLP • Semantic Search • Natural Language Inference")


# ------------------------------------------------
# HERO
# ------------------------------------------------

st.markdown(
    """
    <div class="hero">
        <div class="section-label" style="color:#b9ccff;">
            AI-POWERED RESEARCH ASSISTANT
        </div>
        <h1>Separate claims from evidence.</h1>
        <p>
            Investigate claims using semantic retrieval and natural
            language inference. Explore the available evidence before
            drawing conclusions.
        </p>
    </div>
    """,
    unsafe_allow_html=True,
)


# ------------------------------------------------
# PAGE 1: FACT CHECKER
# ------------------------------------------------

if page == "Fact Checker":

    st.markdown("### Check a claim")
    st.write(
        "Enter a factual statement. The system will search its current "
        "evidence collection and evaluate relevant passages."
    )

    with st.form("fact_check_form"):
        claim = st.text_area(
            "Claim to investigate",
            placeholder=(
                "Example: Human activities are causing the current "
                "global warming."
            ),
            height=125,
            max_chars=2000,
        )

        submitted = st.form_submit_button(
            "🔍 Investigate claim",
            type="primary",
            use_container_width=True,
        )

    if submitted:
        claim = claim.strip()

        if len(claim) < 10:
            st.warning("Please enter a more detailed factual claim.")
```

```
        else:
            st.session_state["last_claim"] = claim

            try:
                with st.spinner(
                    "Searching evidence and evaluating relevant passages..."
                ):
                    checker = get_fact_checker()

                    captured_output = io.StringIO()

                    with contextlib.redirect_stdout(captured_output):
                        with contextlib.redirect_stderr(captured_output):
                            result = checker.check_claim(claim)

                    output = captured_output.getvalue()

                    st.session_state["last_output"] = output
                    st.session_state["last_result"] = (
                        result if result is not None else None
                    )
                    st.session_state["last_checked"] = (
                        datetime.now().strftime("%d %b %Y, %I:%M %p")
                    )

            except Exception as exc:
                st.error("The fact-checking pipeline encountered an error.")
                st.exception(exc)

    if st.session_state.get("last_claim"):
        st.divider()

        st.markdown("### Investigation results")
        st.caption(
            "Claim: " + st.session_state["last_claim"]
        )

        if st.session_state.get("last_checked"):
            st.caption(
                "Checked on " + st.session_state["last_checked"]
            )

        result = st.session_state.get("last_result")
        output = st.session_state.get("last_output", "")

        if isinstance(result, dict):
            verdict = (
                result.get("verdict")
                or result.get("label")
                or result.get("status")
            )

            if verdict:
                normalized = str(verdict).upper()

                if "INSUFFICIENT" in normalized:
                    st.warning("⚠️ " + str(verdict))
                elif "CONTRADICTED" in normalized:
                    st.error("⚠️ " + str(verdict))
                elif "SUPPORTED" in normalized:
                    st.success("✓ " + str(verdict))
                elif "MIXED" in normalized:
                    st.info("Evidence appears mixed.")

            st.markdown("#### Evidence analysis")
```

```
            evidence_items = result.get("evidence", [])

            if evidence_items:
                st.caption(
                    "Only verified evidence meeting the configured "
                    "similarity threshold can influence the verdict."
                )

                used_items = [
                    item for item in evidence_items
                    if item.get("used_for_verdict", False)
                ]

                col1, col2, col3 = st.columns(3)
                col1.metric("Passages retrieved", len(evidence_items))
                col2.metric("Used in verdict", len(used_items))
                col3.metric(
                    "Similarity threshold",
                    f"{result.get('min_similarity', 0.45):.2f}"
                )

                for index, item in enumerate(evidence_items, start=1):
                    title = item.get("title") or "Untitled source"
                    publisher = item.get("publisher") or "Publisher unknown"
                    url = item.get("url") or ""
                    status = item.get("evidence_status", "unknown")
                    similarity = item.get("similarity")
                    nli = item.get("nli") or {}
                    used = item.get("used_for_verdict", False)

                    badge = (
                        "USED FOR VERDICT"
                        if used else "NOT USED FOR VERDICT"
                    )

                    with st.expander(
                        f"{index}. {title} — {badge}"
                    ):
                        st.write("**Publisher:**", publisher)
                        st.write("**Review status:**", status)

                        if url.startswith("https://"):
                            st.markdown(f"[Open source]({url})")

                        if similarity is not None:
                            st.write(
                                f"**Semantic similarity:** {similarity:.4f}"
                            )

                        st.write(
                            "**Used in verdict:**",
                            "Yes" if used else "No"
                        )

                        if not used:
                            if status != "verified":
                                st.caption(
                                    "Excluded because the record is not verified."
                                )
                            elif similarity is not None and similarity < result.get(
                                "min_similarity", 0.45
                            ):
                                st.caption(
                                    "Excluded because similarity is below "
                                    "the configured threshold."
```

```
                            )
                        else:
                            st.caption(
                                "Excluded by the evidence eligibility rules."
                            )

                    st.markdown("**NLI scores**")
                    score_col1, score_col2, score_col3 = st.columns(3)

                    for column, label, key in [
                        (score_col1, "Entailment", "entailment"),
                        (score_col2, "Contradiction", "contradiction"),
                        (score_col3, "Neutral", "neutral"),
                    ]:
                        score = nli.get(key)
                        column.metric(
                            label,
                            f"{score:.1%}"
                            if isinstance(score, (int, float))
                            else "N/A"
                        )

                    text = item.get("text", "")
                    if text:
                        st.markdown("**Evidence snippet**")
                        st.write(text)

            st.caption(
                "Similarity measures relevance, not truth. NLI scores "
                "are model estimates and do not establish factual certainty."
            )
        else:
            st.info("No evidence passages were returned.")

        with st.expander("Technical backend response"):
            st.json(result)

    elif result is not None:
        st.markdown("#### Backend response")
        st.write(result)

    if output.strip():
        st.markdown("#### Evidence analysis")
        st.code(output, language="text")

    if not output.strip() and result is None:
        st.info(
            "The backend returned no displayable result. "
            "Check whether check_claim() prints or returns its analysis."
        )

    report = {
        "project": "Veritas - Evidence-Based Claim Verification",
        "claim": st.session_state["last_claim"],
        "checked_at": st.session_state.get("last_checked"),
        "backend_result": result,
        "analysis_output": output,
        "note": (
            "Automated model output is not proof of truth. "
            "Review source quality, context, and independent evidence."
        ),
    }

    try:
        report_json = json.dumps(
            report,
```

```
                ensure_ascii=False,
                indent=2,
                default=str,
            )

            st.download_button(
                "⬇️ Download investigation report (JSON)",
                data=report_json,
                file_name="fact_check_report.json",
                mime="application/json",
                use_container_width=True,
            )
        except Exception as exc:
            st.caption(f"Report export unavailable: {exc}")

    else:
        st.markdown("### How to use the fact checker")

        col1, col2, col3 = st.columns(3)

        with col1:
            st.markdown(
                """
                <div class="info-card">
                    <h4>01 · Enter a claim</h4>
                    Describe a factual statement you want to investigate.
                </div>
                """,
                unsafe_allow_html=True,
            )

        with col2:
            st.markdown(
                """
                <div class="info-card">
                    <h4>02 · Retrieve evidence</h4>
                    The semantic retrieval component searches stored
                    documents for relevant passages.
                </div>
                """,
                unsafe_allow_html=True,
            )

        with col3:
            st.markdown(
                """
                <div class="info-card">
                    <h4>03 · Review the result</h4>
                    The NLI model evaluates the relationship between
                    the claim and retrieved evidence.
                </div>
                """,
                unsafe_allow_html=True,
            )

        st.info(
            "The current knowledge base may not contain evidence for "
            "every topic. An unsupported claim is not automatically false."
        )


# ------------------------------------------------
# PAGE 2: EVIDENCE STORE
# ------------------------------------------------

elif page == "Evidence Store":
```

```
    st.markdown("### Evidence collection")
    st.write(
        "Browse the documents currently available to the fact-checking engine."
    )

    records = get_evidence_records()

    if not records:
        st.warning("No evidence records were found in the evidence store.")

    else:
        search_text = st.text_input(
            "Search evidence",
            placeholder="Search titles, publishers, or topics...",
        )

        statuses = sorted({
            str(r.get("evidence_status", "unknown"))
            for r in records
        })

        selected_status = st.selectbox(
            "Filter by review status",
            ["All statuses"] + statuses,
        )

        filtered_records = []

        for record in records:
            searchable = " ".join([
                str(record.get("title", "")),
                str(record.get("publisher", "")),
                str(record.get("topic", "")),
            ]).lower()

            if search_text.lower() not in searchable:
                continue

            status = str(record.get("evidence_status", "unknown"))

            if (
                selected_status != "All statuses"
                and status != selected_status
            ):
                continue

            filtered_records.append(record)

        st.caption(
            f"Showing {len(filtered_records)} of {len(records)} records"
        )

        for index, record in enumerate(filtered_records, start=1):
            title = record.get("title") or "Untitled source"
            publisher = record.get("publisher") or "Publisher unknown"
            topic = record.get("topic") or "Uncategorized"
            status = record.get("evidence_status", "unknown")
            url = record.get("url", "")
            text = record.get("text", "")

            with st.expander(f"{index}. {title}"):
                st.write(f"**Publisher:** {publisher}")
                st.write(f"**Topic:** {topic}")
                st.write(f"**Review status:** {status}")
```

```
                if record.get("published_date"):
                    st.write(
                        f"**Publication date:** "
                        f"{record['published_date']}"
                    )

                if url:
                    st.markdown(f"**Source:** [{url}]({url})")

                if text:
                    st.markdown("**Stored text / abstract**")
                    st.write(text[:4000])

                    if len(text) > 4000:
                        st.caption(
                            "Text shortened for display."
                        )

        st.download_button(
            "⬇️ Export evidence metadata (JSON)",
            data=json.dumps(
                filtered_records,
                ensure_ascii=False,
                indent=2,
                default=str,
            ),
            file_name="evidence_store_export.json",
            mime="application/json",
        )


# ------------------------------------------------
# PAGE 3: ABOUT
# ------------------------------------------------

elif page == "About Project":

    st.markdown("### About this project")

    st.write(
        """
        **Veritas: Fake News Detection Using Natural Language Processing
        and Evidence-Based Claim Verification**
        """
    )

    st.markdown("#### Project objectives")

    st.markdown(
        """
        - Investigate factual claims using an evidence collection.
        - Retrieve semantically relevant passages.
        - Apply Natural Language Inference (NLI) to assess relationships
          between claims and evidence.
        - Display source information and model output.
        - Communicate uncertainty rather than automatically labelling
          every claim as true or false.
        """
    )

    st.markdown("#### Technology stack")

    technologies = [
        ("Programming language", "Python"),
        ("Frontend", "Streamlit"),
        ("Semantic retrieval", "Sentence Transformers"),
```

```
        ("Inference model", "DeBERTa-based NLI cross-encoder"),
        ("Evidence sources", "Wikipedia and OpenAlex scholarly records"),
        ("Storage", "JSON Lines (JSONL)"),
    ]

    for technology, description in technologies:
        st.markdown(f"- **{technology}:** {description}")

    st.markdown("#### System workflow")

    st.code(
        "User Claim\n"
        "    ↓\n"
        "Semantic Evidence Retrieval\n"
        "    ↓\n"
        "Relevant Evidence Passages\n"
        "    ↓\n"
        "Natural Language Inference (NLI)\n"
        "    ↓\n"
        "Evidence Analysis and Source Review",
        language="text",
    )

    st.warning(
        "Research limitation: NLI scores are model predictions, not "
        "probabilities that a claim is true. The current evidence store "
        "is limited, some records remain pending review, and the system "
        "does not independently establish source credibility. Results "
        "must be interpreted alongside source quality, publication date, "
        "context, and corroborating evidence."
    )


# ------------------------------------------------
# FOOTER
# ------------------------------------------------

st.divider()

st.caption(
    "Veritas · NLP-Based Evidence Analysis · "
    "Research prototype, not an authoritative fact-checking service."
)
```